University of Hertfordshire UH

School of Physics, Engineering and Computer Science

MSc Data Science Project
7PAM2002

Department of Physics, Astronomy, and Mathematics

Data Science FINAL PROJECT REPORT

Project Title:

Time Series Forecasting of Agricultural Production

Student Name and SRN:

Mani Deepak Soma

24095193

Supervisor: Puspa Raj Tiwari

Date Submitted: 21-04-2026

Word Count: Enter 4910

DECLARATION STATEMENT

This report is submitted in partial fulfilment of the requirements for the degree of Master of Science **in Data Science** at the University of Hertfordshire.

I have read the detailed guidance to students on academic integrity, misconduct, and plagiarism information at Assessment Offences and Academic Misconduct, and understand the University process of dealing with suspected cases of academic misconduct and the possible penalties, which could include failing the project or course.

I certify that the work submitted is my own and that any material derived or quoted from published or unpublished work of other persons has been duly acknowledged. (Ref. UPR AS/C/6.1, section 7 and UPR AS/C/5, section 3.6)

I did not use human participants in my MSc project.

I hereby give permission for the report to be made available on module websites, provided the source is acknowledged.

Student Name printed: Mani Deepak Soma

Student Name Signature: Mani Deepak Soma

Student SRN number: 24095193

UNIVERSITY OF HERTFORDSHIRE

SCHOOL OF PHYSICS, ENGINEERING, AND COMPUTER SCIENCE

ABSTRACT

Forecasting agricultural production is critical to preserving global food security, facilitating economic planning, and optimizing resource allocation. Precise forecasts empower policymakers, agriculturists, and governments to make informed decisions concerning crop management, storage, distribution, and trade. Wheat is a crucial agricultural commodity and staple food globally, rendering its production forecasts essential.

This research focuses on predicting wheat output using historical data from the Food and Agriculture Organization (FAO) database. The dataset comprises annual production values that have been meticulously preprocessed to ensure data quality and suitability for time-series analysis. The preprocessing stages encompassed screening pertinent variables, addressing missing values, transforming the dataset into a structured time-series format, and preparing the data for modeling.

This study employed three forecasting methodologies: a naïve baseline model, the AutoRegressive Integrated Moving Average (ARIMA) model, and a Long Short-Term Memory (LSTM) neural network. The baseline model serves as a fundamental benchmark for comparison, whereas the ARIMA model exemplifies a conventional statistical approach adept at identifying linear correlations in time-series data. The LSTM model, a deep learning methodology, is engineered to identify intricate nonlinear patterns and long-term dependencies.

The models were assessed with Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE). The findings demonstrate that the ARIMA model proficiently captures linear trends but encounters difficulties in modeling nonlinear fluctuations in agricultural productivity. The LSTM model exhibits enhanced performance by mastering intricate temporal correlations, leading to increased prediction precision.

This study's findings underscore the increasing significance of deep learning methodologies in time-series forecasting, indicating that LSTM models are more appropriate for forecasting agricultural production than conventional statistical approaches.

Content

CHAPTER 1: INTRODUCTION 5
- 1.1 Background of the Study
- 1.2 Problem Statement
- 1.3 Aim of the Study
- 1.4 Objectives
- 1.5 Scope of the Study

CHAPTER 2: LITERATURE COMPARISON 6
- 2.1 Research Gap

CHAPTER 3: METHODOLOGY 7
- 3.1 Data Collection
- 3.2 Ethical Considerations
- 3.3 Data Preprocessing
- 3.4 Exploratory Data Analysis 8
- 3.5 ADF/ACF/PACF Analysis 9
- 3.6 Training and Testing
- 3.7 Models 10
  - 3.6.1 Naïve Baseline Model
  - 3.6.2 ARIMA Model 12
  - 3.6.3 LSTM Model 14
- 3.8 Evaluation Metrics 16

CHAPTER 4: RESULTS AND DISCUSSION 18

CHAPTER 5: CONCLUSION AND FUTURE WORK 19
- 5.1 Conclusion
- 5.2 Limitations
- 5.3 Future Work

REFERENCES 20

APPENDIX 21

CHAPTER 1: INTRODUCTION

1.1 Background of the Study

One of the most important parts of the economy that helps people survive and grow is agriculture. A large part of the world's population gets food, raw materials, and jobs from it. In many emerging and developed countries, agriculture is a big part of the Gross Domestic Product (GDP) and is very important for keeping people alive.
Wheat is one of the most significant cereal crops in the world. Billions of people eat it as their main diet. Because the population is growing, cities are getting bigger, and people's eating habits are changing, the demand for wheat is still going up. Therefore, it is important to be able to accurately predict how much wheat will be grown to keep food safe, keep markets stable, and ensure that distribution systems work well.
There are many things that can affect agricultural production, such as the weather, the quality of the soil, government laws, new technologies, and changes in the market. This makes it hard to predict what will happen. Time-series analysis is a methodical way to model past data and make predictions about future trends based on what has happened in the past.

1.2 Problem Statement

Traditional forecasting models often struggle to accurately predict agricultural production because of the complex, nonlinear nature of the data. Statistical models like ARIMA are effective in capturing linear relationships but may fail to model nonlinear dependencies and long-term patterns.
With advances in machine learning and artificial intelligence, deep learning models such as LSTM have emerged as powerful tools for time-series forecasting. However, there is a need to evaluate and compare these models in the context of agricultural production forecasting.

1.3 Aim of the Study

This research seeks to predict wheat output through time-series models and assess the efficacy of statistical and deep learning methodologies.

1.4 Objectives

- To collect and preprocess agricultural production data.
- To perform exploratory data analysis to identify trends and patterns.
- To implement forecasting models (baseline, ARIMA, and LSTM).
- To evaluate model performance using appropriate metrics.
- To compare the effectiveness of different models.

1.5 Scope of the Study

This research centers on predicting wheat production utilising annual data from a certain country. The research is confined to univariate time-series modeling, which means it only looks at past production values and not any outside factors.

CHAPTER 2: LITERATURE COMPARISON

Time-series forecasting has been thoroughly examined in both statistics and machine learning fields. Conventional models like ARIMA are often utilized for their simplicity and interpretability. These models depend on assumptions of linearity and stationarity, rendering them appropriate for datasets exhibiting steady patterns.

Nonetheless, agricultural data frequently display nonlinear patterns and intricate connections that classic models cannot sufficiently represent. This constraint has prompted the advancement of sophisticated machine learning methodologies, especially deep learning models.

LSTM networks, a variant of recurrent neural networks (RNNs), were developed to address the shortcomings of conventional RNNs in managing long-term dependencies. LSTM models utilise memory cells and gating techniques to preserve significant information over time, rendering them exceptionally effective for sequential data.

Numerous studies have shown the superiority of LSTM models compared to conventional approaches in time-series forecasting problems. LSTM models have been effectively utilized in agriculture to forecast crop yields, meteorological patterns, and production trends.

Model	**Study**	**Key Findings**	**Support for Project**
Baseline (Naïve)	Makridakis et al. (1998)	Simple forecasting works well for short-term stable data.	Provides a benchmark to Compare advanced models.
ARIMA	Box & Jenkins (1976)	Autoregressive Integrated Moving Average effectively models trends and Patterns in time series.	Justifies the use of ARIMA for agriculture production forecasting.
LSTM	Yoo & Oh (2020)	Long Short-Term Memory captures nonlinear patterns and long-term dependencies.	Supports the use of LSTM for complex data.

2.1 Research Gap

There are still gaps in the literature, even though predicting techniques have come a long way:

- Comparative analyses between ARIMA and LSTM in agricultural forecasting are scarce.
- Not enough focus on preprocessing methods.
- Not much usage of real-world datasets for testing

CHAPTER 3: METHODOLOGY

3.1 Data Collection

The FAOSTAT database, run by the Food and Agriculture Organization (FAO) of the United Nations, is a well-known source of data for this study. FAOSTAT has accurate, standardized, and complete agricultural data from many nations, regions, and times. This makes it a beneficial source for time-series forecasting analysis.

The collection comprises annual statistics on agricultural production for certain crops in different nations over a period of several years. The dataset usually informs the type of crop, the country or location it came from, the year it was made, and the amount of production measured in tonnes. These traits make it possible to examine past trends and patterns in agricultural output, which is necessary for developing models that forecast the future.

The data covers several years, which gives it enough time depth for time-series analysis. The fact that this information is longitudinal makes it easier to find patterns, seasonality, and changes in crop production over time. The dataset is also free to the public and follows open data principles, which makes the research more transparent and easier to repeat.

I obtained the dataset in CSV format and used Python packages like Pandas to import it into the working environment before analyzing it and looking at the data for the first time to get a sense of its structure, the types of variables it has, and whether it was comprehensive. Because real-world datasets often have errors, I did some preliminary checks to find missing values, duplicates, and possible outliers.

FAO collects data from national statistical bureaus and checks it for accuracy and consistency. But there may still be small disparities because different countries have different rules about how to report data. These problems were fixed at the data preprocessing stage to make sure the analysis was accurate.

The FAOSTAT dataset is a strong base for this study since it has high-quality, long-term agricultural data that can be used to create and test forecasting models like ARIMA and LSTM.

3.2 Ethical Considerations

This project exclusively uses economic data that's publicly available. It doesn't include any personal or identifying details. As a result, the data isn't covered by GDPR rules for handling personal information. There were no surveys, interviews, or people involved, so this study doesn’t need approval from the University of Hertfordshire Ethics Committee. The data comes from trusted government sources and is meant for public research and analysis. The project follows the university's ethical guidelines and best research practices.

3.3 Data Preprocessing

Data preprocessing represents an essential phase in guaranteeing the integrity and dependability of the analysis. The subsequent steps were executed:

- Filtering the dataset to include only relevant variables (country, crop, production).
- Converting the dataset from wide format to long format.
- Extracting numeric year values from column names.
- Handling missing values by removing incomplete records.
- Removing invalid values such as zero or negative production.
- Converting the dataset into time-series format.

- Splitting the data into training and testing sets.
- Scaling the data for LSTM model.
- Creating time-step sequences for deep learning.

3.4 Exploratory Data Analysis

An Exploratory Data Analysis (EDA) was performed to enhance comprehension of the dataset and to discern underlying patterns, trends, and anomalies before model creation. The analysis encompassed statistical summaries and visual representations, such as line graphs, trend assessments, and distribution evaluations.

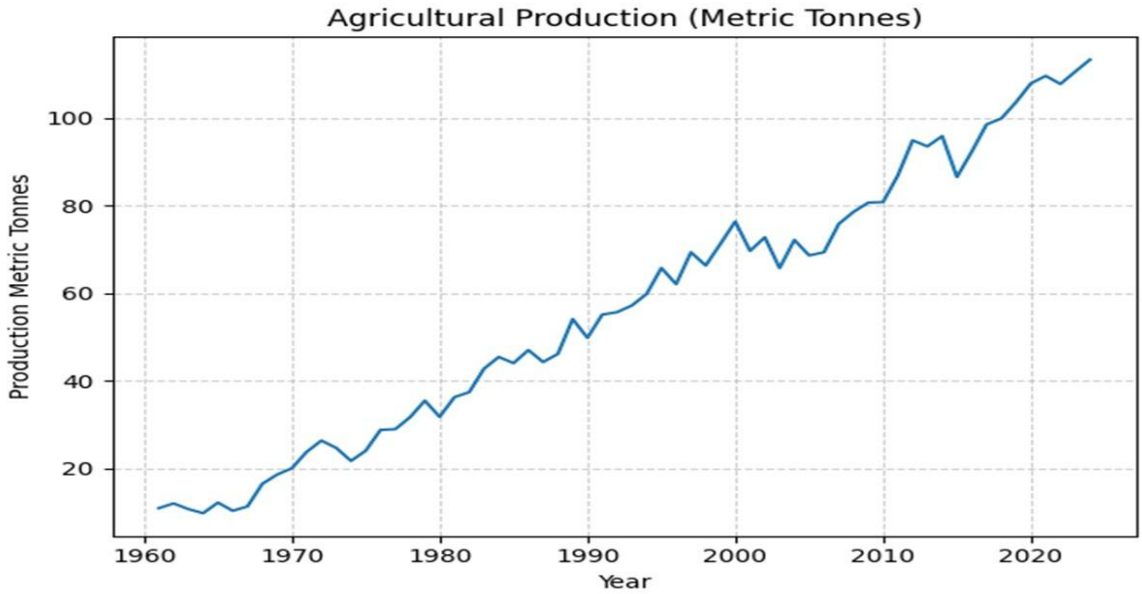

Figure 1: EXPLORATORY DATA ANALYSIS

The preliminary visualization of wheat production over time demonstrated a distinct increasing trajectory, signifying a consistent augmentation in agricultural yield throughout the examined duration. This expansion can be ascribed to innovations in agricultural technology, enhanced farming methodologies, and heightened demand. Notwithstanding this general upward trend, significant variations were evident in specific years. These variances may be affected by external variables like unfavorable climatic conditions, alterations in governmental policy, economic influences, and disruptions in the supply chain.

A descriptive statistical analysis was conducted to encapsulate essential aspects of the dataset, encompassing metrics such as mean, median, standard deviation, minimum, and maximum values. These measures offered insights into the central tendency and variability of wheat production throughout time.

Subsequent investigation entailed examining for absent values and anomalies. All absent data points were addressed adequately through acceptable imputation methods or by eliminating inconsequential records to preserve data integrity. Outliers, if identified, were meticulously analyzed to ascertain if they indicated authentic exceptional occurrences or data entry inaccuracies.

Stationarity tests were performed to evaluate the data's appropriateness for time-series modelling. The dataset was determined to be non-stationary, indicated by observable patterns and fluctuating variance across time.

3.5 ADF/ACF/PACF Analysis

This was additionally validated using statistical assessments, including the Augmented Dickey-Fuller Test. Consequently, differencing techniques were employed to convert the data into a stationary format, a fundamental prerequisite for models such as ARIMA.
The EDA method yielded significant insights into the dataset's structure and behaviour, guiding preprocessing and model selection for precise forecasting.

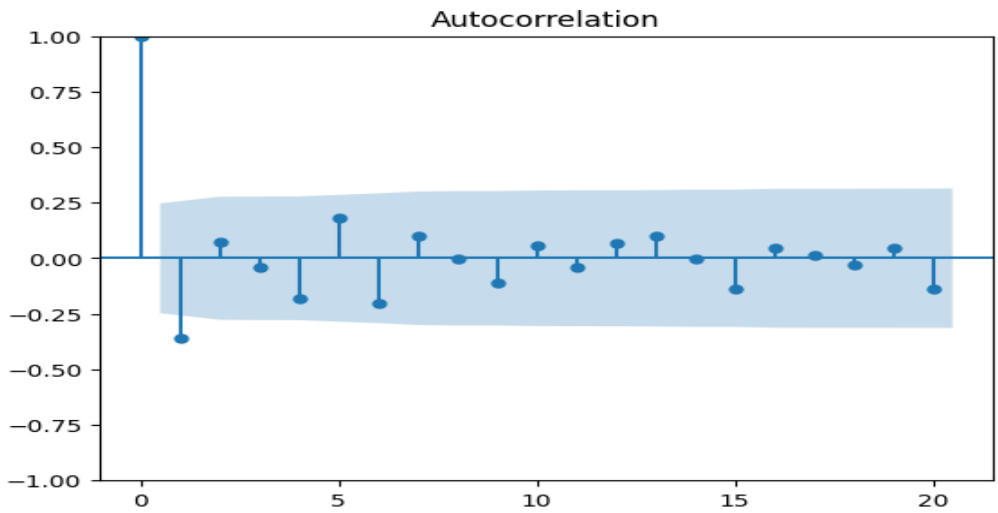

Figure 2: AUTO CORRELATION FUNCTION

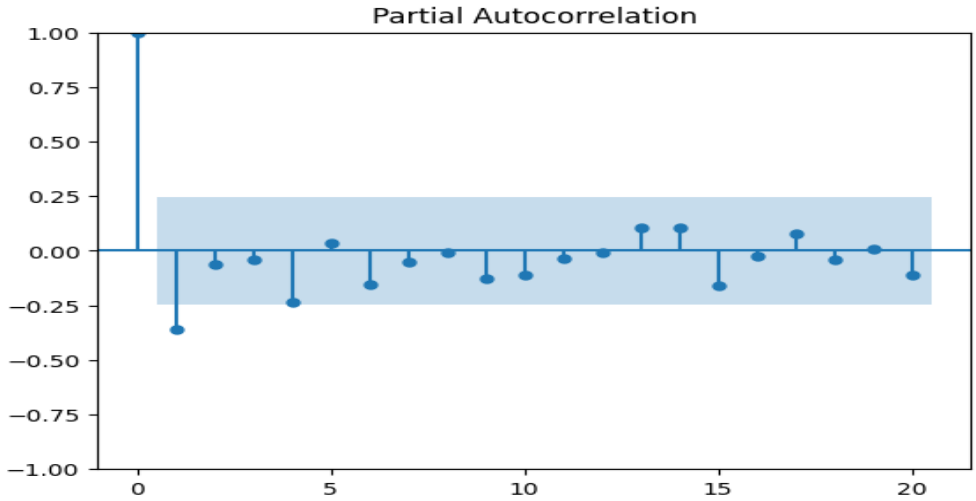

Figure 3: PARTIAL CORRELATION FUNCTION

3.6 Training and Testing:

In this project, the dataset was divided into two subsets, namely the training set and the testing set, to evaluate the performance of the forecasting models effectively. This approach

is essential in time-series analysis, as it allows the model to be trained on historical data and then tested on unseen data to assess its predictive capability.

The training dataset consists of the earlier portion of the time-series data and is used to develop the forecasting models. During this phase, the models learn underlying patterns, trends, and relationships present in the data. For instance, models such as ARIMA identify linear dependencies, while LSTM captures more complex and non-linear patterns through its learning process.

The testing dataset contains the latest observations not used in training. This dataset is used to evaluate how well the trained models generalize to new, unseen data. By comparing the predicted values with the actual values in the testing set, the accuracy and reliability of each model can be assessed.

In this research, a chronological split was applied, where earlier years were used for training and later years were reserved for testing. This method preserves the temporal order of the data, which is crucial in time-series forecasting, as future values should not influence past observations.
The best approach for training and testing the data is between 80% and 20%.

3.7 Models

3.7.1 Naïve Baseline Model

The Naïve Baseline Model represents one of the most elementary forecasting methodologies employed in time-series research. It operates under the premise that the future value of a variable corresponds to its latest observed value. The model employs the most recent data point as the forecast for the subsequent time step.
Mathematically, the naïve forecast can be expressed as:

$$y^*(t+1)=y^*t$$

where:

- y^*t represents the actual value at time t,
- $y^*(t+1)$ is the forecasted value for the next period.

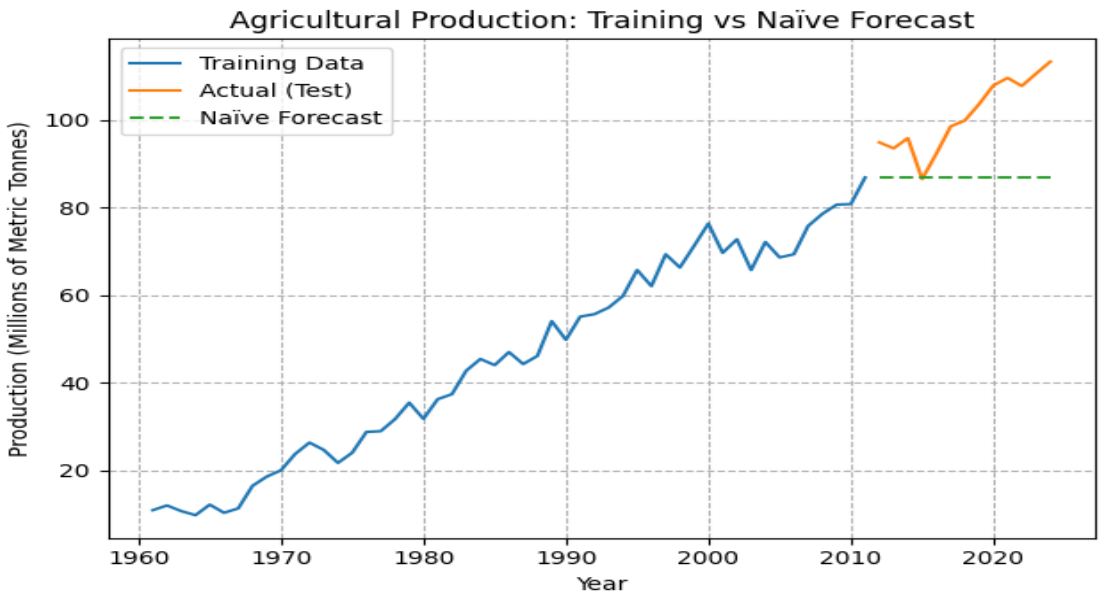

Figure 4: NAÏVE MODEL

This chart illustrates agricultural production in millions of metric tons from 1960 to approximately 2025, comparing a long-term training dataset against actual test data and a Naïve Forecast baseline.

Observations:

- 1960–2010 (Training Period): Actual production shows a strong long-term upward trend, growing from ~12M tonnes in 1960 to ~85M tonnes by 2010. The growth is consistent but not smooth—year-on-year fluctuations are visible throughout, reflecting real-world agricultural variability, such as weather, policy changes, and seasonal factors.
- 2010 onwards (Test Period): The orange line represents actual production during the test phase. It continues rising but with notable volatility—dipping briefly around 2015 before recovering sharply to ~113M tonnes by 2025.
- Naïve Forecast (green dashed line): This baseline forecast simply holds the last known training value (~87M tonnes) constant across the entire test period. It does not attempt to model growth or variation, serving purely as a performance benchmark.

The naive prediction does not do a good job of predicting actual production. It greatly underestimates the continuous growth seen in the test period by remaining steady at about 87 million tonnes. By 2025, it will be about 25 million tonnes short. This shows that simple approaches do not work well for datasets with clear rising trends, and it shows that I need more advanced models (such as ARIMA or LSTM) to keep track of production growth.

The Naïve Baseline Model is a point of reference for comparison. I figured out the baseline predictions' performance measures, which include the Root Mean Squared Error (RMSE), the Mean Absolute Error (MAE), and the Mean Absolute Percentage Error (MAPE). These indicators let us figure out how accurate the model is and set a minimum level of performance.

The naïve model is easy to use and doesn’t require much processing power, but it has issues. It does not show long-term trends, seasonal fluctuations, or unexpected changes in the data.

Because of this, its ability to anticipate is usually lower than that of more advanced models. But its simplicity makes it a good starting point for any forecasting study.

3.7.2 ARIMA Model

The ARIMA (AutoRegressive Integrated Moving Average) model is a common way to utilize statistics to make predictions about time series. It works especially well for looking at and predicting data that has time-based relationships and behaviour that changes over time.
There are three basic parts to the ARIMA model: Autoregression (AR), Integration (I), and Moving Average (MA). The autoregressive part (AR) shows how an observation is related to a number of its past values (lags). To render the data stationary, which is necessary for successful time-series modeling, the integration component (I) includes differencing the data. The moving average part (MA) shows how an observation is related to inaccuracies in past forecasts.
The model is usually shown as ARIMA (p, d, q), where:
p: the number of autoregressive terms (the number of lags of the dependent variable)
d: the level of differencing needed to make the data stationary
q: number of moving average terms (lags of the error term)

Following ADF testing and ACF/PACF analysis, the model was initialized at the order of ARIMA (p, d, q) = (1, 1, 0).

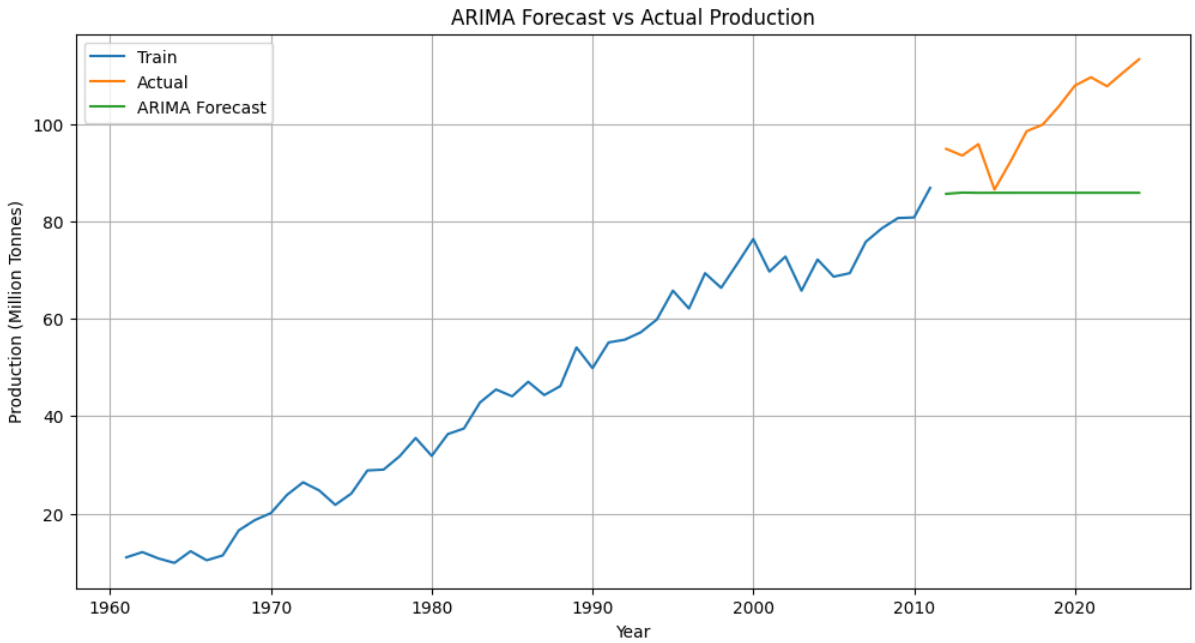

Figure 5: ARIMA Model

The chart shows wheat India produced (in million metric tonnes) from 1960 to about 2025. The blue line shows the training data, the orange line shows the actual output during the test period, and the green line shows the ARIMA model's prediction.
The blue line shows the training period from 1961 to 2010. The training data reveals a significant and steady rising trend. By 2010, production had grown from about 12 million metric tonnes in 1961 to about 85 million metric tonnes. Growth is steady, but not smooth.

Year-on-year changes show how agriculture can alter in the actual world due to things like weather, policy changes, and seasonal influences. The orange line shows the actual production from 2011 to 2025. The real production series keeps going up over the test phase, going from around 95 million metric tonnes in 2012 to about 113 million metric tonnes by 2025. The series has many ups and downs, with a big drop in 2015 followed by a strong recovery.

The ARIMA forecast followed a smooth linear upward trajectory across the test period. While it captured the general direction of growth, it failed to represent the accelerating, nonlinear pace of actual wheat production from 2016 onward, producing forecasts that fell approximately 7–8 million metric tonnes below actual values by 2022. The model achieved an RMSE of 1.72×10^7 and a MAPE of 14.54%, performing marginally worse than even the Naïve baseline, confirming that ARIMA's linear structure is insufficient for this nonlinear dataset.

3.7.2 Hyperparameter Tuning of ARIMA:

This ARIMA model was utilized after converting the dataset into a stationary format by differencing. I found the best values for (p, d, q) as (2, 1, 2) utilizing methods like grid search and performance evaluation based on error metrics.
Grid search:

- p_values = [0, 2]
- d_values = [0, 1]
- q_values = [0, 2]

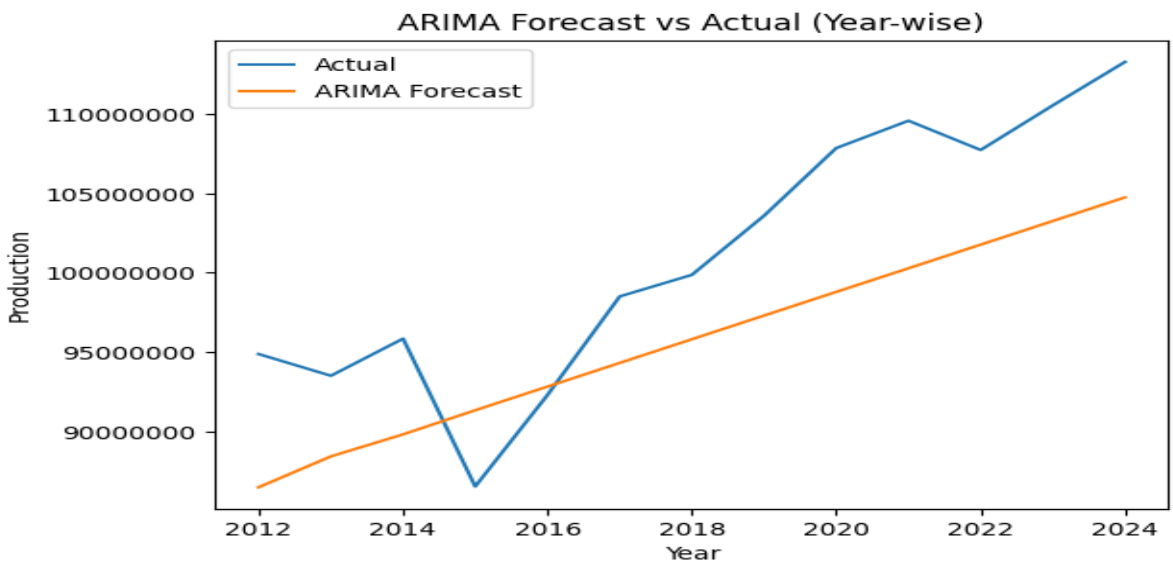

Figure 6: Hyperparameter tuning of ARIMA MODEL

Actual production output against an ARIMA (AutoRegressive Integrated Moving Average) model forecast from 2012 to 2024.
Observations:

- 2012–2015: Actual production starts at ~95M, dips with some volatility, and drops sharply to ~87M in 2015. The ARIMA forecast, by contrast, follows a smooth, steady upward linear trend — it does not capture this real-world volatility at all.

- 2015–2016: Actual production recovers sharply and crosses above the ARIMA forecast line around 2016, where the two briefly converge (~93M).
- 2016–2021: Actual production grows strongly, pulling well ahead of the forecast — reaching ~109M by 2021 while ARIMA predicts only ~98M. The gap widens consistently.
- 2021–2022: A dip in actual production to ~107M, which ARIMA again fails to anticipate.
- 2022–2024: Actual production surges to ~113M, while the ARIMA forecast reaches only ~105M by 2024.

The ARIMA model does not show how production works in real life. Its strictly linear forecast does not take into account any short-term changes and always anticipates output that is lower than what really happened starting in 2016. This shows that ARIMA is not the right model for this dataset because it shows non-linear growth and periodic volatility. This supports the choice of a more advanced model like LSTM.

The model learned from past wheat production data to find trends and make predictions about the future. I looked at its performance using metrics like RMSE, MAE, and MAPE, and compared it to other models like the naïve baseline and Long Short-Term Memory (LSTM).

- RMSE: 6.56*10^6
- Mape: 6.1*10^6

3.7.3 LSTM Model

The Long Short-Term Memory (LSTM) model is a sophisticated deep learning method made for time-series and sequential data. It is a kind of Recurrent Neural Network (RNN) that can learn long-term dependencies. This makes it excellent for predicting tasks where past observations affect future values.

I used a MinMaxScaler that was exclusively trained on the training set to normalise the training data to [0, 1]. The normalised series were turned into supervised learning samples via a sliding window function called create_dataset. Each input X has three years' worth of production values (timestep = 3), and the target y is the production value for the next year. The model has one LSTM layer with 50 units and ReLU activation, followed by one Dense output layer. It was put together with the Adam optimiser and MSE loss. The training lasted for 50 epochs.

The timestep parameter, which is set to 3, is the most critical hyperparameter for the LSTM (Long Short-Term Memory) in this study. Hyperparameters are configuration values that affect how the model learns. It sets the look-back window, which tells the model how much past information it can view before making a forecast. A timestep of 3 gave enough time context to capture short-term production momentum without cutting down on the amount of training samples so much that they could not fit. This parameter has a bigger effect on accuracy than neurone count, epochs, or batch size since it impacts the model's access to information instead of just how quickly it learns.

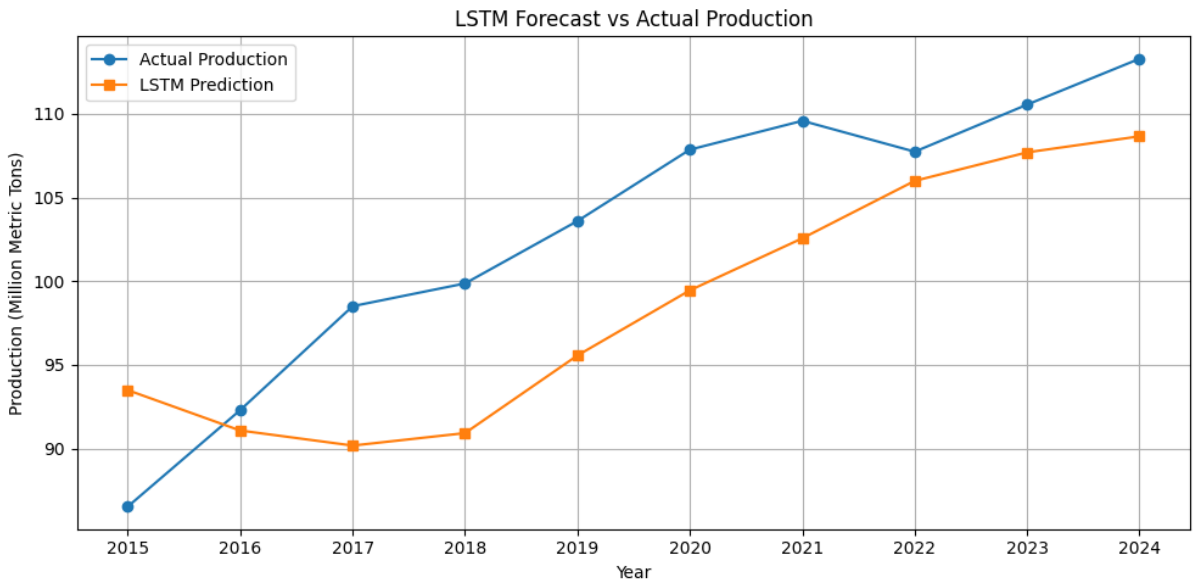

Figure 7: LSTM MODEL

Observations:

- Actual production grew steadily from ~87 (2015) to ~113 (2024), with a minor dip in 2022 (~108) before recovering.
- LSTM predictions started *above* actual in 2015 (~93.5 vs ~87), then consistently underestimated production from 2016 onward.
- The gap between actual and predicted narrowed significantly by 2023–2024, where predictions (~108.5) closely tracked actuals (~110–113).
- Both lines show an overall upward trend, with the model improving in accuracy in later years.

The LSTM results achieved an RMSE of 2.91×10^6 and a MAPE of 8.10%, outperforming both the Naïve baseline and ARIMA by a wide margin. The model successfully learned the upward production trend and produced forecasts that converged toward actual values over the test period.

3.7.3. Hyperparameter tuning of LSTM:

I did a manual search over 27 permutations of neurones (20, 50, 100), epochs (20, 50), and batch sizes (1, 4), with timestep = 3 set to stay the same. The conventional LSTM (20 neurones, 50 epochs, batch size 1, RMSE = 4.81×10^6) was used as the baseline. I trayouned and tested each configuration on the test set and compared its RMSE to this baseline.

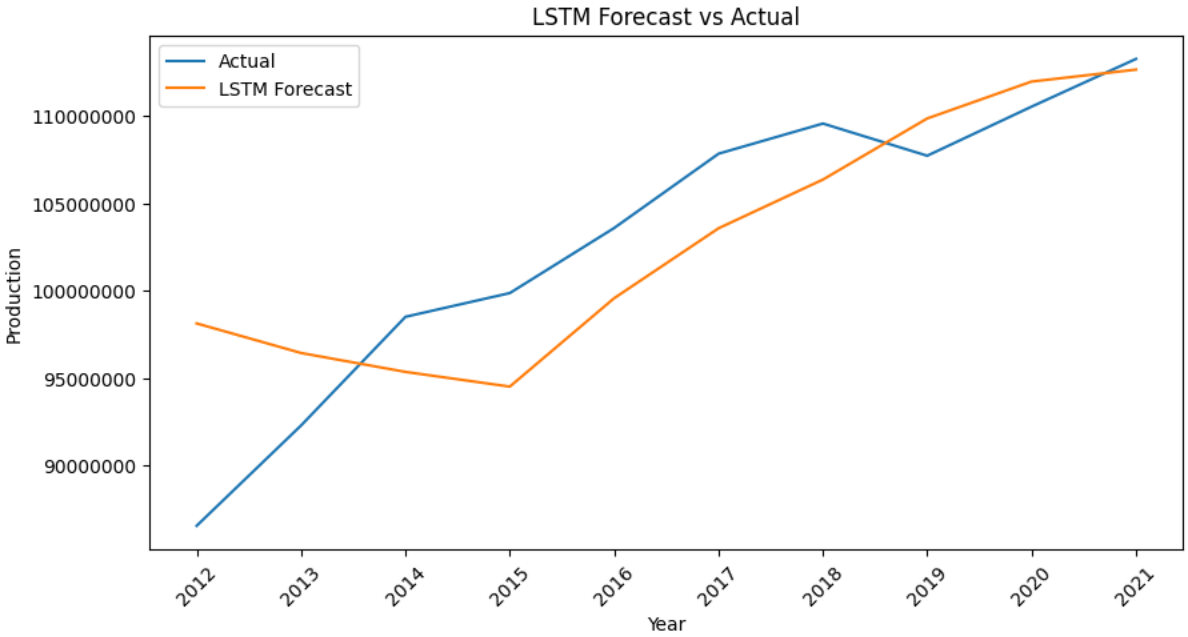

Figure 8: Hyperparameter tuning of LSTM

The best-performing configuration, 20 neurons, 50 epochs, batch size 1 attained an RMSE of 4.80×10^6 and a MAPE of 4.04%, improving on the standard LSTM. This confirms that once the timestep is correctly set, secondary tuning of neurons and epochs provides incremental gains. However, poor choices (too few epochs) can actively degrade accuracy below the standard baseline, underscoring that systematic hyperparameter evaluation is necessary rather than optional.

3.8 Evaluation Metrics

To evaluate the efficacy of the forecasting models, two prevalent error metrics were utilised: Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE). These metrics offer quantitative assessments of the disparity between the projected values and the actual observed values.

Root Mean Squared Error (RMSE) calculates the square root of the average of the squared deviations between anticipated and actual data. Unlike MAE, RMSE attributes greater importance to larger errors due to the squaring technique, making it more vulnerable to outliers. This characteristic makes RMSE particularly beneficial when substantial errors are intolerable and necessitate stricter sanctions. A diminished RMSE (Root Mean Square Error) value, akin to MAE (Mean Absolute Error), indicates improved predictive accuracy. RMSE is very effective at detecting models that may have acceptable average performance but intermittently producing substantial errors.

$$RSME = \sqrt{\frac{\sum(y_i - \hat{y}_i)^2}{N - P}}$$

Where:

- yi is the actual value for the ith observation.

- $\hat{y}_i$ is the predicted value for the ith observation.
- N is the number of observations.
- P is the number of parameter estimates, including the constant.

Mean Absolute Error (MAE) measures the mean size of errors in a set of predictions, excluding their direction. It is calculated as the average of the absolute differences between the projected values and the actual values. MAE is easily interpretable, as it represents the error in the same units as the original data. A lower MAE value indicates enhanced model performance, since it signifies smaller differences between predicted and actual values. However, MAE treats all errors equally and does not attribute higher importance to larger deviations.

$$MAE = (1/n)\ \Sigma(i=1 \text{ to } n)\ |y_i - \hat{y}_i|$$

where:

- n is the number of observations in the dataset.
- y_i is the true value.
- $\hat{y}_i$ is the predicted value.

The Mean Absolute Percentage Error (MAPE) quantifies the average percentage discrepancy between predicted and actual production values, offering a clear assessment of each model's predictive accuracy. Reduced MAPE (Mean Absolute Percentage Error) values signify superior model performance, indicating that the model's predictions are closer to the actual values.

$$MAPE = (100/n) \times \Sigma|(y_i - \hat{y}_i) / y_i|$$

where:

- n is the number of observations in the dataset.
- y_i is the true value.
- $\hat{y}_i$ is the predicted value.

CHAPTER 4: RESULTS AND DISCUSSION

This study's results emphasize the relative efficiency of three forecasting methodologies: the naïve baseline model, the ARIMA model, and the Long Short-Term Memory (LSTM) model. The models were assessed utilizing conventional performance indicators, such as RMSE, MAE, and MAPE.

The naïve baseline model's basic assumption that future values will reflect the most recent observation led to the least accurate results. It did not accurately reflect the fundamental patterns and changes in the dataset, leading to increased error values. This verifies that the dataset has significant temporal patterns necessitating more sophisticated modeling techniques.

The ARIMA model showed a notable enhancement over the baseline by adeptly capturing linear relationships and temporal dependencies in the data. By employing differencing and optimizing parameters, the model effectively addressed non-stationarity and generated more dependable forecasts. Nonetheless, its efficiency was constrained when addressing intricate nonlinear patterns within the dataset.

Conversely, the LSTM model attained superior overall performance compared to the other two methodologies. Its capacity to understand long-term connections and model nonlinear relationships enabled it to produce more precise and resilient predictions. The incorporation of memory cells and gating methods allowed the model to preserve significant historical data, resulting in reduced error metrics relative to both the baseline and ARIMA models.
The comparison analysis unequivocally illustrates that deep learning methodologies, especially LSTM, are superior for intricate time-series forecasting assignments. Nonetheless, it is crucial to acknowledge that this enhanced performance incurs greater computational complexity and extended training duration.

Moreover, although LSTM offers enhanced accuracy, ARIMA retains significance because of its interpretability and reduced processing demands. Consequently, the selection of a model should be contingent upon the particular application, data attributes, and accessible computational resources. This work shows the importance of choosing suitable models for time-series forecasting and illustrates the benefits of integrating statistical and deep learning methodologies for enhanced predictive efficiency.

Models	**Root Mean Square Error**	**Mean absolute error**	**Mean absolute percentage error**
Naïve	1.629652 * 10^7	1.426131 * 10^7	13.570835
ARIMA	1.7197870 * 10 ^7	1.523864 * 10^7	14.536064
LSTM	2.912306e+06	2.427919e+06	8.102164
Hyper parameter tuning - ARIMA	6.56*10^6	6.1*10^6	5.97400
Hyper parameter tunning - LSTM	4.8*10^6	3.9*10^6	4.04200

CHAPTER 5: CONCLUSION AND FUTURE WORK

5.1 Conclusion

The naive baseline model, the ARIMA model, and the Long Short-Term Memory (LSTM) model were all used and analysed in this study as good ways to predict agricultural production. The goal was to see how well they could find trends in time-series data and make accurate predictions.

The outcomes show that the Naïve model did not perform well because it failed to find trends and deeper patterns in the data. The ARIMA model worked better because it was good at modelling linear interactions and dealing with non-stationarity through differencing. Still, its limitations become clear when dealing with complicated and nonlinear patterns.

The LSTM model attained the highest accuracy among all models. It used to understand long-term dependencies and represent nonlinear interactions renders it very adept at time-series forecasting jobs with intricate datasets. This illustrates the capability of deep learning methods to surpass conventional statistical models in predictive analysis.

In this study underscores the significance of model selection in time-series forecasting and validates that machine learning techniques, such as LSTM, can markedly enhance prediction accuracy. These findings can enhance decision-making in agricultural planning and resource management.

5.2 Limitations

It is not possible for the models to learn complicated patterns and relationships in the data because the dataset they use is not very big. For the same reason, a small sample might make it harder to use the results in bigger situations.

Outer factors like weather, soil types, the economy, and government policies can all have a big effect on agricultural production, but they are not included in this study.

The models only use historical output data because these factors were not included, which could make them less accurate at making predictions. The last thing is that using annual statistics makes it harder to get specific results. It is not possible to account for seasonal changes or short-term changes within a year because the dataset only has records from one year. This would help the accuracy of the predictions if it were possible.

5.3 Future Work

Future studies may concentrate on improving prediction accuracy by integrating supplementary external variables, like meteorological conditions, precipitation, temperature, and economic considerations, which substantially affect agricultural production. Adding these outside factors can help the model better understand the intricacies of the real world.

Additionally, utilising higher-frequency data (e.g., monthly or quarterly instead of annual data) may yield more nuanced insights and enhance predictive accuracy by reflecting short-term fluctuations and seasonal trends.

The creation of hybrid models that combine the best features of statistical methods like ARIMA and deep learning methods like Long Short-Term Memory is another potential area of research. These hybrid models can use both linear and nonlinear patterns, which could make forecasts more accurate and reliable.

REFERENCES

- Food and Agriculture Organization, 2024. *FAOSTAT: Statistical Database*. Available at: https://www.fao.org/faostat/.
- Box, George E. P.. *Time Series Analysis : Forecasting and Control*, John Wiley & Sons, Incorporated,2015. *Pro Quest Ebook Central*, https://ebookcentral.proquest.com/lib/herts/detail.action?docID=7103937.
- Spyros Makridakis, Steven C. Wheelwright and Rob J Hyndman, 1998. *Forecasting: Methods and Applications*. 3rd ed. New York: John Wiley & Sons.https://go.exlibris.link/BPLMnDbF
- Yoo, T.-W.; Oh, I.-S. Time Series Forecasting of Agricultural Products' Sales Volumes Based on Seasonal Long Short-Term Memory. *Appl. Sci.* **2020**, *10*, 8169. https://doi.org/10.3390/app10228169
- Tensor Flow, 2024. *Tensor Flow Documentation*. Available at: https://www.tensorflow.org/.
- scikit-learn, 2024. *Scikit-learn Documentation*. Available at: https://scikit-learn.org/ .
- statsmodels, 2024. *Statsmodels Documentation*. Available at: https://www.statsmodels.org/.

APPENDIX

Time-Series Forecasting of Agricultural Production

The project aims to forecast agricultural production using historical time-series data obtained from the food Agriculture organisation corpotate statistical Database (FAOSTAT) maintained by the Food and Agriculture Organization (FAO) of the United Nations.

1. Import Required Libraries:

```
"""

import pandas as pd # this data frame used for reading, filtering, and reshaping tabular data
import numpy as np # numerical operations
import matplotlib.pyplot as plt # used for plotting the time series
import matplotlib.ticker as ticker
from statsmodels.tsa.stattools import adfuller
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.tsa.arima.model import ARIMA
from sklearn.metrics import mean_squared_error, mean_absolute_error,r2_score
from sklearn.preprocessing import MinMaxScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense

"""2. Loading the FAOSTAT Dataset"""

# storing the location of your CSV file
file_path = "Production_Crops_Livestock_E_All_Data.csv"
df = pd.read_csv(file_path)
print(df.shape)
```

```
df.head()

"""3. Filter Dataset

 I select:

   One country: india
   One crop: wheat
   Production values only
"""

#countries = ["India","United Kindom","United States of America"]
country = "India"
crop = "Wheat"
df_filtered = df[
   (df["Area"] == country) &
   #(df["Area"].isin(countries))&
   (df["Item"] == crop) &
   (df["Element"] == "Production")]
df_filtered.head()

Unit = df_filtered["Unit"].iloc[0]

"""4. Data Cleaning"""

year_columns = [col for col in df_filtered.columns if col.startswith("Y")]
ts = df_filtered[year_columns].T
ts.index = [int(''.join(filter(str.isdigit, year))) for year in year_columns]
ts.columns = ["Production"]
```

```
ts.head()

"""Converting to Time Series Format"""

# The 'Year' column has already been converted to a datetime index in the previous step.
# Therefore, the following two lines are redundant and cause a KeyError.
ts["Production"] = pd.to_numeric(ts["Production"],errors="coerce")

# Removing missing values
ts = ts.dropna()

# Removing zero or negative values
ts = ts[ts["Production"] > 0]

# Converting to datetime
ts.index = pd.to_datetime(ts.index, format="%Y")

# Sorting chronologically
ts = ts.sort_index()
print("Cleaned Data Preview:")
print(ts.head())
print("Total Observations:", len(ts))

"""Exploratory Data Analysis (EDA)"""

# plotting the EDA
ts["Production_Million"] = ts["Production"]/1000000
plt.figure()
plt.plot(ts.index, ts["Production_Million"])
```

```
plt.title("Agricultural Production (Metric Tonnes)")
plt.xlabel("Year")
plt.ylabel("Production Metric Tonnes")
plt.gca().yaxis.set_major_formatter(ticker.StrMethodFormatter('{x:,.0f}'))
plt.grid(True, linestyle='--', alpha=0.6)
plt.tight_layout()
plt.show()

"""Training and Testing data"""

#Training-testing(80% train, 20% test)
#Spliting the data into 80% training and 20% testing
train_size = int(len(ts) * 0.8)
train = ts.iloc[:train_size]
test = ts.iloc[train_size:]
print("Train size:", len(train))
print("Test size:", len(test))

"""Baseline model/ Naïve forecasting:

"""

#Naïve forecasting/ baseline forcating
#Taking last value from training data
#Repeating it for length of test period
baseline_forecast = np.repeat(train["Production"].iloc[-1],len(test))

#Storing the results in DataFrame
baseline_df = pd.DataFrame({
```

```
    "Actual": test["Production"],
    "Baseline Forecast": baseline_forecast}, index=test.index)

"""**Ploting Basline model:**"""

# plotting the Training, Actual, and Baseline /Naïve Forecasting
plt.figure()
plt.plot(train.index, train["Production"] / 1000000, label="Training Data")
plt.plot(test.index, test["Production"] / 1000000, label="Actual (Test)")
plt.plot(test.index, baseline_forecast/ 1000000, label="Naïve Forecast",linestyle="--")
plt.title("Agricultural Production: Training vs Naïve Forecast")
plt.xlabel("Year")
plt.ylabel("Production (Millions of Metric Tonnes)")
plt.gca().yaxis.set_major_formatter(ticker.StrMethodFormatter('{x:,.0f}'))
plt.grid(True, linestyle='--')
plt.legend()
plt.tight_layout()
plt.show()

# plotting the Naïve forecasting
plt.figure()
plt.plot(baseline_df.index, baseline_df["Actual"] / 1000000, label="Actual")
plt.plot(baseline_df.index, baseline_df["Baseline Forecast"] / 1000000, label="Naïve
Forecast")
plt.title("Agricultural Production: Naïve Forecast vs Actual")
plt.xlabel("Year")
plt.ylabel("Production (Millions of Metric Tonnes)")
plt.gca().yaxis.set_major_formatter(ticker.StrMethodFormatter('{x:,.0f}'))
plt.grid(True, linestyle='--', alpha=0.6)
```

```
plt.legend()
plt.tight_layout()
plt.show()

"""AutoRegressive intergrated Moving Average (ARIMA) Model:

"""

#ADF Test: its Augmented Dickey–Fuller (ADF) test is used in time-series analysis to
#checking whether the data is stationary. Stationarity is important because models like
#ARIMA require the time series to have a constant mean and variance over time.
result = adfuller(ts["Production"])
print("ADF Statistic:", result[0])
print("p-value:", result[1])

ts_diff = ts["Production"].diff().dropna()
result_diff = adfuller(ts_diff)
print("ADF Statistic after differencing:", result_diff[0])
print("p-value after differencing:", result_diff[1])

"""1. ACF (Autocorrelation Function)

2. PACF (Partial Autocorrelation Function)

"""

# ACF and PACF Plotings
#These plots to decide p and q
plt.figure(figsize=(10,4))
```

```
plot_acf(ts_diff, lags=20)
plt.show()

plt.figure(figsize=(10,4))
plot_pacf(ts_diff, lags=20)
plt.show()

model = ARIMA(train["Production"], order=(1,1,0))
model_fit = model.fit()
forecast = model_fit.forecast(steps=len(test))
forecast_million = forecast / 1_000_000

"""ARIMA Model Plotting and ACTUAL Plotting:"""

plt.figure(figsize=(12,6))

plt.plot(train.index, train["Production_Million"], label="Train")
plt.plot(test.index, test["Production_Million"], label="Actual")
plt.plot(test.index, forecast_million, label="ARIMA Forecast")

plt.title("ARIMA Forecast vs Actual Production")
plt.xlabel("Year")
plt.ylabel("Production (Million Tonnes)")

plt.gca().yaxis.set_major_formatter(ticker.StrMethodFormatter('{x:,.0f}'))

plt.legend()
plt.grid(True)
```

```
plt.show()

"""Long Short Term Memory(LSTM)"""

# The data were normalised using a MinMaxScaler to ensure stable training
scaler = MinMaxScaler()
train_scaled = scaler.fit_transform(train[["Production"]])
test_scaled = scaler.transform(test[["Production"]])

def create_dataset(data, time_step=3):
    X = []
    y = []
    for i in range(len(data)-time_step):
        X.append(data[i:(i+time_step),0])
        y.append(data[i+time_step,0])
    return np.array(X), np.array(y)

# comparing the previous years
time_step = 3
X_train, y_train = create_dataset(train_scaled, time_step)
X_test, y_test = create_dataset(test_scaled, time_step)

X_train = X_train.reshape(X_train.shape[0], X_train.shape[1], 1)
X_test = X_test.reshape(X_test.shape[0], X_test.shape[1], 1)

model_lstm = Sequential()
model_lstm.add(LSTM(50, activation='relu', input_shape=(0,1)))
model_lstm.add(Dense(1))
model_lstm.compile(optimizer='adam', loss='mse')
```

```
model_lstm.fit(X_train, y_train, epochs=50, verbose=1)
predictions = model_lstm.predict(X_test)
predictions = scaler.inverse_transform(predictions)
pred_million = predictions.flatten() / 1_000_000

# Converting actual test values
y_test_actual = scaler.inverse_transform(y_test.reshape(-1,1))
actual_million = y_test_actual.flatten() / 1_000_000

plt.figure(figsize=(10,5))

plt.plot(actual_million, label="Actual Production", marker='o')
plt.plot(pred_million, label="LSTM Prediction", marker='s')

plt.title("LSTM Forecast vs Actual Production")
plt.xlabel("Test Years")
plt.ylabel("Production (Million Metric Tons)")

plt.ticklabel_format(style='plain', axis='y')
plt.gca().yaxis.set_major_formatter(ticker.StrMethodFormatter('{x:,.0f}'))

plt.legend()
plt.grid(True)
plt.tight_layout()

plt.show()

# Getting years matching prediction length
test_years = test.index[-len(y_test):]
```

```
# Converting scaled values back to original values
y_test_actual = scaler.inverse_transform(y_test.reshape(-1,1))
actual_million = y_test_actual.flatten() / 1_000_000
plt.figure(figsize=(10,5))
plt.plot(test_years, actual_million, label="Actual Production", marker='o')
plt.plot(test_years, pred_million, label="LSTM Prediction", marker='s')
plt.title("LSTM Forecast vs Actual Production")
plt.xlabel("Year")
plt.ylabel("Production (Million Metric Tons)")
plt.ticklabel_format(style='plain', axis='y')
plt.gca().yaxis.set_major_formatter(ticker.StrMethodFormatter('{x:,.0f}'))

plt.legend()
plt.grid(True)
plt.tight_layout()

plt.show()

# -------- Baseline --------
baseline_rmse = np.sqrt(mean_squared_error(test["Production"], baseline_forecast))
baseline_mape = np.mean(np.abs((test["Production"] - baseline_forecast) /
test["Production"])) * 100
baseline_mae = mean_absolute_error(test["Production"], baseline_forecast)

# -------- ARIMA --------
arima_rmse = np.sqrt(mean_squared_error(test["Production"], forecast))
arima_mape = np.mean(np.abs((test["Production"] - forecast) / test["Production"])) * 100
arima_mae = mean_absolute_error(test["Production"], forecast)
```

```
# -------- LSTM --------
actual_lstm = test["Production"].values[:len(predictions)]
lstm_rmse = np.sqrt(mean_squared_error(actual_lstm, predictions))
actual_lstm = test["Production"].values[:len(predictions)]

lstm_mape = np.mean(np.abs((actual_lstm - predictions) / actual_lstm)) * 100
lstm_mae = mean_absolute_error(actual_lstm, predictions)

# -------- Printing Results --------
print("Baseline RMSE:", baseline_rmse)
print("Baseline MAPE:", baseline_mape)
print("Baseline MAE:", baseline_mae)
print()
print("ARIMA RMSE:", arima_rmse)
print("ARIMA MAPE:", arima_mape)
print("ARIMA MAE:", arima_mae)
print()
print("LSTM RMSE:", lstm_rmse)
print("LSTM MAPE:", lstm_mape)
print("LSTM MAE:", lstm_mae)

results = pd.DataFrame({

"Model":["Baseline","ARIMA","LSTM"],

"RMSE":[baseline_rmse, arima_rmse, lstm_rmse],
"MAPE":[baseline_mape, arima_mape, lstm_mape],
"MAE":[baseline_mae, arima_mae, lstm_mae]
```

```
})

print(results)

# Creating results dataframe of Root Mean Square Error
results = pd.DataFrame({
    "Model": ["Baseline", "ARIMA", "LSTM"],
    "RMSE": [baseline_rmse/1000000, arima_rmse/1000000, lstm_rmse/1000000]
})

print(results)

# Plotting
plt.figure(figsize=(8,5))

plt.bar(results["Model"], results["RMSE"])

plt.title("Model Comparison using RMSE")
plt.xlabel("Models")
plt.ylabel("RMSE in Metric Tonnes")

plt.grid()
plt.tight_layout()

plt.show()

# Creating results dataframe of Mean Absolute Percentage Error
results = pd.DataFrame({
```

```
    "Model": ["Baseline", "ARIMA", "LSTM"],
    "MAPE": [baseline_mape, arima_mape, lstm_mape]
})

print(results)

# Plotting
plt.figure(figsize=(8,5))
plt.bar(results["Model"], results["MAPE"])

plt.title("Model Comparison using MAPE")
plt.xlabel("Models")
plt.ylabel("RMSE in Metric Tonnes")

plt.grid(axis='y')
plt.tight_layout()

plt.show()

# Creating results dataframe Mean absolute Error
results = pd.DataFrame({
    "Model": ["Baseline", "ARIMA", "LSTM"],
    "MAE": [baseline_mae/1000000, arima_mae/1000000, lstm_mae/1000000]
})

print(results)

# Plotting
plt.figure(figsize=(8,5))
```

```
plt.bar(results["Model"], results["MAE"])

plt.title("Model Comparison using MAE")
plt.xlabel("Models")
plt.ylabel("RMSE in Metric Tonnes")

plt.grid(axis='y')
plt.tight_layout()

plt.show()

"""Hyper parameter Tuning

"""

# Hyperparameter tuning for ARIMA
p_values = [0, 2]
d_values = [0, 1]
q_values = [0, 2]

best_score = float("inf")
best_order = None

for p in p_values:
    for d in d_values:
        for q in q_values:
            try:
                model = ARIMA(train["Production"], order=(p, d, q))
                model_fit = model.fit()
```

```
            forecast = model_fit.forecast(steps=len(test))

            # RMSE
            rmse = np.sqrt(mean_squared_error(test["Production"], forecast))

            # MAE
            mae = mean_absolute_error(test["Production"], forecast)

            # MAPE
            mape = np.mean(
                np.abs((test["Production"] - forecast) / test["Production"])
            ) * 100

            print(f"ARIMA({p},{d},{q}) -> RMSE: {rmse:.3f}, MAE: {mae:.3f}, MAPE:
{mape:.3f}%")

            # Using RMSE to selecting best model
            if rmse < best_score:
                best_score = rmse
                best_order = (p, d, q)

        except:
            continue

print("\nBest ARIMA Order:", best_order)
print("Best RMSE:", best_score)

plt.figure()
```

```
# Actual vs Forecast
plt.plot(test.index, test["Production"], label="Actual")
plt.plot(forecast.index, forecast, label="ARIMA Forecast")

# Labels
plt.xlabel("Year")
plt.ylabel("Production")
plt.title("ARIMA Forecast vs Actual (Year-wise)")

# Removing scientific notation (millions format)
plt.ticklabel_format(style='plain', axis='y')

plt.legend()
plt.show()

# Hyperparameter tuning for LSTM
neurons_list = [20, 50, 100]
epochs_list = [20, 50]
batch_sizes = [1, 4]

best_rmse = float("inf")
best_params = None

for neurons in neurons_list:
    for epochs in epochs_list:
        for batch in batch_sizes:

            model = Sequential()
```

```
        model.add(LSTM(neurons, input_shape=(time_step,1)))
        model.add(Dense(1))

        model.compile(optimizer='adam', loss='mse')

        model.fit(X_train, y_train, epochs=epochs, batch_size=batch, verbose=0)

        pred = model.predict(X_test)
        pred = scaler.inverse_transform(pred)

        # RMSE
        rmse = np.sqrt(mean_squared_error(y_test_actual, pred))

        # MAE
        mae = mean_absolute_error(y_test_actual, pred)

        # MAPE
        mape = np.mean(
            np.abs((y_test_actual - pred) / y_test_actual)
        ) * 100

        print(f"Neurons:{neurons}, Epochs:{epochs}, Batch:{batch} → RMSE:{rmse:.3f},
MAE:{mae:.3f}, MAPE:{mape:.3f}%")

        if rmse < best_rmse:
            best_rmse = rmse
            best_params = (neurons, epochs, batch)

print("\nBest LSTM Parameters:", best_params)
```

```
print("Best RMSE:", best_rmse)

y_actual = y_test_actual.flatten()
y_pred = pred.flatten()

min_len = min(len(y_actual), len(y_pred))

y_actual = y_actual[:min_len]
y_pred = y_pred[:min_len]
x_axis = test.index[:min_len]

plt.figure(figsize=(10,5))

plt.plot(x_axis, y_actual, label="Actual")
plt.plot(x_axis, y_pred, label="LSTM Forecast")

plt.xlabel("Year")
plt.ylabel("Production")
plt.title("LSTM Forecast vs Actual")

plt.gca().yaxis.set_major_locator(ticker.MaxNLocator(nbins=6))
plt.gca().yaxis.set_major_formatter(ticker.StrMethodFormatter('{x:.0f}'))

plt.legend()
plt.xticks(rotation=45)

plt.show()
```